

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Parallel Computing

Báo cáo thực hành

Shock Wave Blast Simulation with Work Pool

GVHD: Diệp Thanh Đăng
La Quốc Nhật Huân

Sinh viên: Hà Chí Trung - 2213682

TP HỒ CHÍ MINH, THÁNG 12 2025

1 Mở đầu

1.1 Dẫn nhập

Vào ngày 3 tháng 9 năm 2025, Trung Quốc đã tổ chức một cuộc diễu hành Ngày Chiến thắng đánh dấu kỷ niệm 80 năm chiến thắng Nhật Bản và kết thúc Thế chiến II. Đáng chú ý, Trung Quốc lần đầu tiên công bố tên lửa đạn đạo xuyên lục địa (ICBM) DF-61 - được mô tả là một trong những hệ thống tiên tiến nhất của họ - với tầm bắn ước tính từ 12.000 đến 15.000 km. Có tin đồn cho rằng DF-61 được trang bị đầu đạn hạt nhân, mỗi đầu đạn có sức công phá ước tính khoảng 650 kiloton - mạnh hơn gấp 300 lần quả bom "Little Boy" thả xuống Hiroshima.

1.2 Đối tượng nghiên cứu

Trong một vụ nổ hạt nhân, không khí xung quanh bị nén mạnh, tạo ra sóng xung kích lan truyền ra bên ngoài. Sóng này được đặc trưng bởi sự tăng áp suất đột ngột so với áp suất khí quyển bình thường, gọi là áp suất vượt mức (*overpressure*). Giá trị áp suất vượt mức được xác định thông qua khoảng cách tỷ lệ (Scaled Distance) dựa trên định luật Hopkinson-Cranz, được mô tả như sau:

$$Z = R \cdot W^{-1/3}$$

trong đó:

- Z là khoảng cách tỷ lệ ($\text{m}/\text{kg}^{1/3}$),
- R là khoảng cách đến tâm vụ nổ (m),
- W là năng lượng nổ tương đương TNT (kg).

Áp suất vượt mức cực đại (tính bằng kPa) là mức áp suất lớn nhất vượt lên trên áp suất khí quyển và có quan hệ trực tiếp với khả năng gây thiệt hại. Giá trị này được xác định dựa trên các mô hình thực nghiệm (số liệu Kingery-Bulmash). Đầu tiên, ta tính giá trị trung gian U như sau:

$$U = -0.21436 + 1.35034 \cdot \log_{10}(Z)$$

Áp suất vượt mức cực đại sau đó được xác định gián tiếp thông qua phương trình sau, với các hệ số C_i thu được từ thực nghiệm:

$$\log_{10}(P_{so}) = \sum_{i=0}^8 C_i \cdot U^i$$

Trong đó các hệ số:

$$\begin{aligned} C_0 &= 2.611369, \\ C_1 &= -1.690128, \\ C_2 &= 0.00805, \\ C_3 &= 0.336743, \\ C_4 &= -0.005162, \\ C_5 &= -0.080923, \\ C_6 &= -0.004785, \\ C_7 &= 0.007930, \\ C_8 &= 0.000768. \end{aligned}$$

Trong thực tế, sóng xung kích ban đầu lan truyền nhanh hơn tốc độ âm thanh do áp suất vượt mức rất cao và giảm dần khi nó lan rộng. Tuy nhiên, để đơn giản hoá mô phỏng, thời gian sóng lan đến một vị trí được xấp xỉ bằng giả định rằng sóng chuyển động với tốc độ âm thanh. Như vậy, thời gian để sóng đến khoảng cách R được tính bởi:

$$t = \frac{R}{343}$$

1.3 Yêu cầu

Giả sử tất cả các đầu đạn DF-61 được kích nổ tại tâm của một bản đồ được biểu diễn bởi ma trận kích thước 4000×4000 , ban đầu được khởi tạo bằng không; mỗi ô đại diện cho một khu vực có kích thước 10 mét $\times$ 10 mét. Theo ước tính của chuyên gia GPT, vụ nổ có sức công phá xấp xỉ 5000 kiloton. Trong bài thực hành này, sinh viên phải mô phỏng sự lan truyền của sóng xung kích trong 100 giây đầu tiên. Mô phỏng tiến hành theo từng bước thời gian mỗi giây; khi mặt trước của sóng xung kích lan đến một ô, giá trị của ô đó được thay thế bằng giá trị áp suất vượt mức (peak overpressure) tương ứng tại vị trí đó.

Cụ thể, sinh viên được yêu cầu thực hiện các nhiệm vụ sau:

- Phát triển một phiên bản mô phỏng tuần tự (single-threaded) của bài toán.
- Xây dựng phiên bản mô phỏng song song sử dụng mô hình *work pool*.
- Thực hiện so sánh thời gian chạy giữa hai phiên bản tuần tự và song song.

Lưu ý: Mặc dù mô hình *work pool* chủ yếu được thiết kế để quản lý song song theo tác vụ, nó cũng được áp dụng rộng rãi trong các bài toán song song dữ liệu. Mô hình này cho phép phân phối khối lượng công việc hiệu quả giữa các luồng, giúp cân bằng tải và tối ưu hóa việc sử dụng tài nguyên.

2 Thiết kế và cài đặt

2.1 Dữ liệu đầu vào

Dựa trên các công thức lý thuyết về sự lan truyền sóng xung kích và các yêu cầu được nêu trong đề bài, mô phỏng sử dụng các dữ liệu đầu vào như sau:

- **Không gian mô phỏng:** Khu vực xung quanh vụ nổ được biểu diễn dưới dạng một ma trận hai chiều kích thước 4000×4000 . Mỗi phần tử trong ma trận tương ứng với một ô lưới có diện tích 10 m $\times$ 10 m. Toàn bộ ma trận được khởi tạo ban đầu bằng giá trị không.
- **Thông số vật lý:** Các hằng số sử dụng trong mô phỏng bao gồm:
 - Kích thước ô lưới: `CELL_SIZE` = 10 (mét),
 - Tốc độ lan truyền sóng xung kích (xấp xỉ tốc độ âm thanh): `SOUND_SPEED` = 343 (m/s),
 - Năng lượng nổ tương đương TNT: `YIELD` = 5000 (kiloton), được quy đổi thành đơn vị kilogram theo công thức $W = 10^6 \times \text{YIELD}$.
- **Hàm tính áp suất vượt mức:** Hàm `compute_overpressure` được xây dựng dựa trên các mô hình thực nghiệm trong phần lý thuyết, bao gồm bước tính *scaled distance*, giá trị trung gian U , và cuối cùng là giá trị *peak overpressure* thông qua đa thức Kingery-Bulmash.

```
1 double compute_overpressure(double R)
2 {
3     double W = YIELD_KG;
4
5     if (R == 0) return 1e9; // avoid log(0)
6
7     double Z = R * std::pow(W, -1.0/3.0);
8     double U = -0.21436 + 1.35034 * std::log10(Z);
9
10    double C[9] = {
11        2.611369, -1.690128, 0.00805, 0.336743,
12        -0.005162, -0.080923, -0.004785,
13        0.007930, 0.000768
14    };
15
16    double logP = 0.0;
17    double Ui = 1.0;
18
19    for (int i = 0; i < 9; i++) {
20        logP += C[i] * Ui;
21        Ui *= U;
22    }
23
24    return std::pow(10.0, logP);
25 }
```

Những dữ liệu đầu vào này đóng vai trò là cơ sở cho cả hai phiên bản mô phỏng: mô phỏng tuần tự và mô phỏng song song theo mô hình *work pool*.

2.2 Phiên bản tuần tự

Phiên bản tuần tự đóng vai trò là mô hình tham chiếu (baseline) để so sánh với phiên bản song song. Trong bài này, chúng ta sử dụng ô ở vị trí $N / 2 = 2000$ làm trung tâm vụ nổ:

```
1 const int N = GRID_SIZE;
2 const int cx = N / 2;
3 const int cy = N / 2;
```

Mô phỏng được tiến hành theo từng bước thời gian rời rạc, từ $t = 0$ đến $t = 100$ giây, sử dụng một vòng lặp:

```
1 for (int t = 0; t <= SIM_TIME; t++)
```

Ở mỗi bước thời gian t , Tính bán kính tối đa và ô lưới mà sóng xung kích có thể lan tới được:

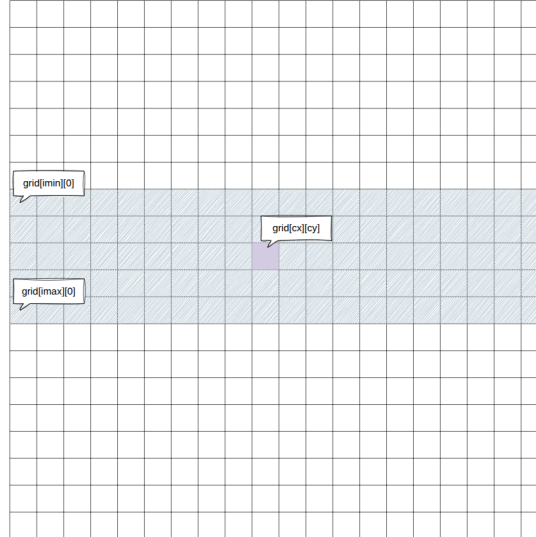
```
1 double Rmax = SOUND_SPEED * t;
2 double Rmax_cell = Rmax / CELL_SIZE;
```

Thay vì duyệt toàn bộ ma trận kích thước 4000×4000 , chúng ta sẽ chỉ xử lý các ô nằm trong vòng tròn có bán kính $R_{\max}^{\text{cell}}$ tính từ tâm vụ nổ bằng cách sau:

- Các hàng cần duyệt tới là khoảng từ $i_{\min}$ đến $i_{\max}$:

```
1 int imin = std::max(0, cx - (int)Rmax_cell);
2 int imax = std::min(N - 1, cx + (int)Rmax_cell);
```

Ví dụ với $Rmax_cell = 2$, $imin = Rmax_cell - 2$ và $imax = Rmax_cell + 2$



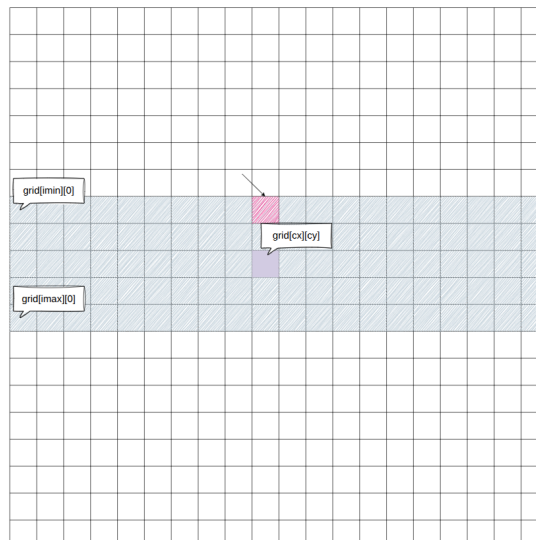
- Tại mỗi hàng các ô cần thực hiện tính toán nằm ở các cột từ $jmin$ đến $jmax$:

```

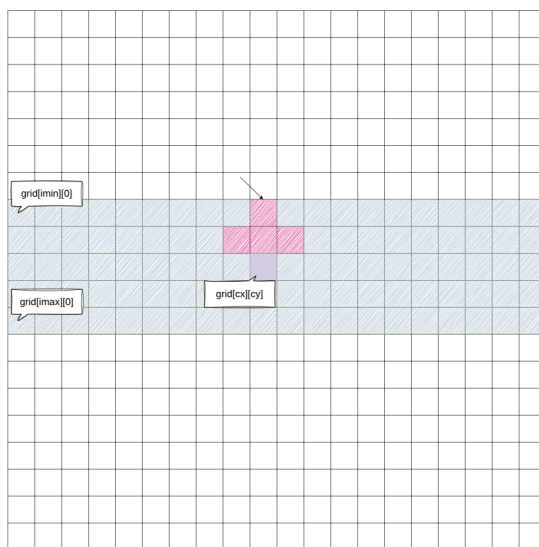
1  double dx_cell = (i - cx);
2  double dy_max_cell = std::sqrt(Rmax_cell * Rmax_cell - dx_cell *
3      dx_cell);
4  int jmin = std::max(0, (int)(cy - dy_max_cell));
5  int jmax = std::min(N - 1, (int)(cy + dy_max_cell));

```

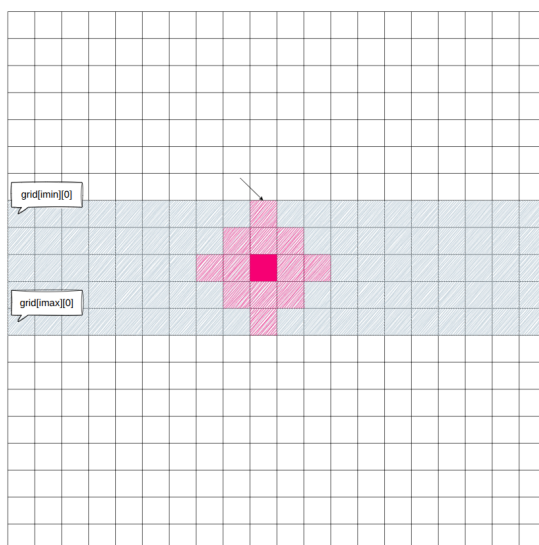
Tiếp tục ví dụ, tại hàng đầu tiên, $jmin = jmax = cy$:



Tại hàng thứ hai, $jmin = cy - 1$ và $jmax = cy + 1$:



Tương tự như vậy cho đến hàng cuối cùng ta sẽ xác định được các ô cần tính toán nằm trong một vùng quanh bán kính như sau:



Cách tiếp cận này giúp giảm đáng kể số lượng phép tính khi thời gian còn nhỏ, đồng thời đảm bảo mô phỏng bám sát mô hình vật lý.

Đối với mỗi ô nằm trong phạm vi ảnh hưởng tại thời điểm t , thuật toán thực hiện các bước sau:

```
1  for (int j = jmin; j <= jmax; j++)  
2  {  
3      if (grid[i][j] > 0.0)  
4          continue;  
5  }
```

```
6         double dx = (i - cx) * CELL_SIZE;  
7         double dy = (j - cy) * CELL_SIZE;  
8  
9         double R = std::sqrt(dx * dx + dy * dy);  
10  
11        double arrival_time = R / SOUND_SPEED;  
12  
13        if (arrival_time <= t)  
14            grid[i][j] = compute_overpressure(R);  
15    }
```

1. Nếu ô đã được cập nhật trước đó, bỏ qua.
2. Tính khoảng cách R từ ô tới tâm nổ.
3. Xác định thời điểm sóng xung kích đến ô:
4. Nếu $t_{\text{arrive}} \leq t$, sóng xung kích đã lan tới vị trí này và giá trị áp suất vượt mức sẽ được tính bằng hàm `compute_overpressure`.
5. Ghi kết quả vào ô.

Việc đảm bảo mỗi ô chỉ được ghi đúng một lần giúp mô phỏng phản ánh đúng hiện tượng vật lý: sóng xung kích chỉ lan tới một vị trí đúng vào thời điểm xác định. Nhờ cách duyệt theo vòng tròn lan rộng tại mỗi bước thời gian, phiên bản tuần tự đạt hiệu suất cao hơn nhiều so với việc quét toàn bộ ma trận tại mọi thời điểm, đồng thời giữ được tính chính xác của mô hình.

2.3 Phiên bản song song

Phiên bản song song của mô phỏng được xây dựng dựa theo mô hình *work pool*. Trong mô hình này, toàn bộ khối lượng tính toán tại mỗi bước thời gian được chia thành nhiều tác vụ nhỏ (tasks). Các tác vụ này được đưa vào một hàng đợi dùng chung, từ đó nhiều luồng worker sẽ lấy tác vụ ra và xử lý một cách độc lập. Cách tiếp cận này cho phép cân bằng tải động và tận dụng tối đa năng lực xử lý của các lõi CPU.

Các bước thực hiện của giải thuật song song gần như tương tự với giải thuật tuần tự đã trình bày phía trên, điểm khác biệt cốt lõi nằm ở sau bước xác định phạm vi các hàng cần tính toán. Sau khi xác định các hàng cần tính toán, chúng ta chia công việc tính toán riêng mỗi hàng thành một task, đẩy chúng vào hàng đợi và các worker sẽ tính toán các bước còn lại tương tự như giải thuật tuần tự:

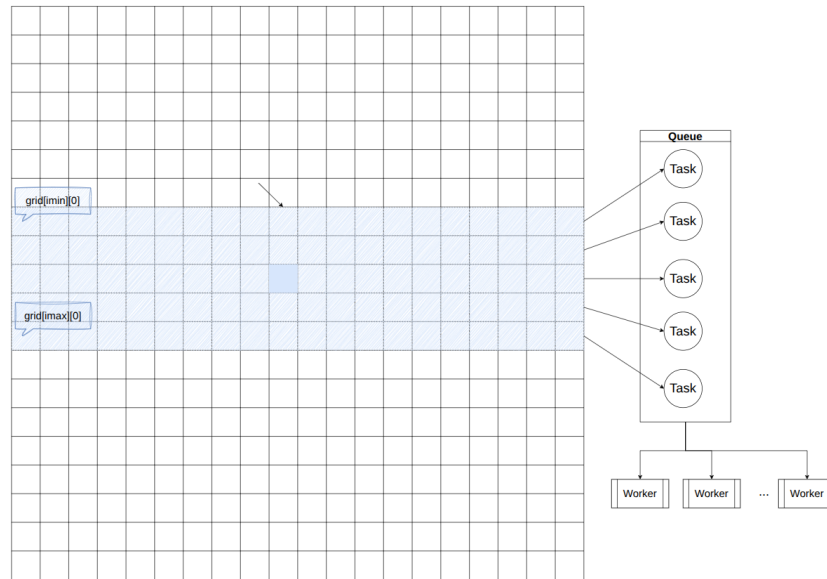


Figure 1: Hình minh họa giải thuật song song với mô hình work pool

Tổng quát, quá trình hoạt động của mô hình work pool như sau:

1. Khởi tạo các worker threads và trạng thái đồng bộ.
2. Duyệt qua từng bước thời gian $t = 0 \dots T$.
3. Tại mỗi t , tính bán kính lan truyền:

$$R_{\max}(t) = v \cdot t, \quad R_{\max}^{cell} = \frac{R_{\max}}{\text{CELL_SIZE}}$$

4. Xác định phạm vi các hàng nằm trong vùng ảnh hưởng:

$$i \in [cx - R_{\max}^{cell}, cx + R_{\max}^{cell}]$$

5. Tạo các task cho từng hàng và đẩy vào hàng đợi:

```
1 for (int i = imin; i <= imax; ++i) {
2     Task task{i, t, Rmax_cell};
3     {
4         std::lock_guard<std::mutex> lock(queue_mutex);
5         task_queue.push(task);
6     }
7 }
8 cv.notify_all();
```

6. Chờ tất cả task của bước thời gian hoàn thành trước khi tiếp tục.
7. Sau khi T bước thời gian kết thúc, đặt cờ dừng:

```
1 g_stop = true;
2 cv.notify_all();
```

8. Join toàn bộ worker và kết thúc.

Cách triển khai này mang lại các lợi ích sau:

- **Cân bằng tải động:** mỗi worker lấy task ngay khi rảnh, không xảy ra tình trạng một luồng bị quá tải trong khi luồng khác nhàn rỗi.
- **Tận dụng tối đa CPU:** workers luôn trong trạng thái bận trừ khi không còn tác vụ.
- **Giảm chi phí đồng bộ:** mỗi task xử lý một hàng độc lập nên không cần khóa cho ma trận.
- **Hiệu năng cao:** số hàng gần tâm có nhiều ô, hàng xa có ít ô; work pool phân phối tự động phù hợp.

3 Kết quả và phân tích

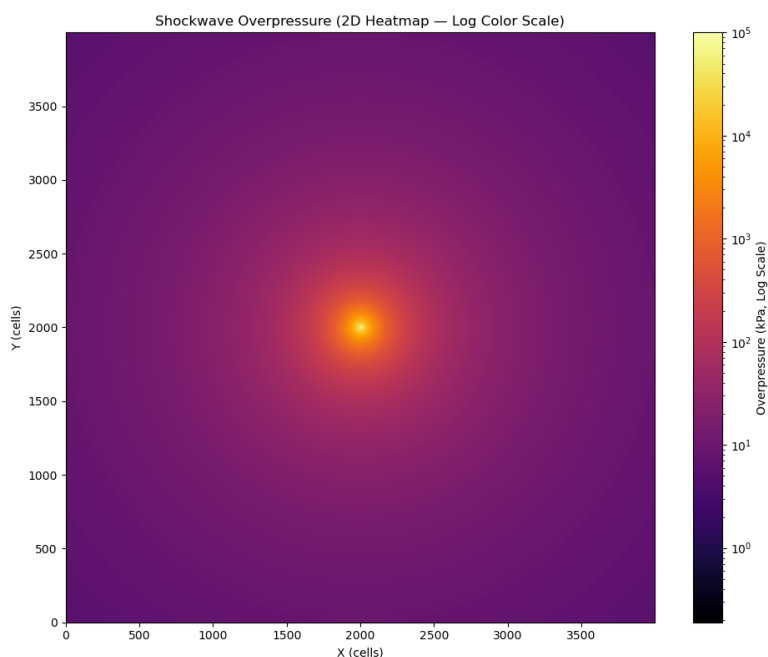


Figure 2: Biểu đồ heatmap overpressure tại thời điểm 100 giây

Dựa trên các kết quả đo thời gian thực thi giữa phiên bản tuần tự và phiên bản song song sử dụng mô hình *work pool*, ta thu được bảng thời gian như sau:

Số luồng (threads)	Thời gian chạy (giây)
1 (tuần tự)	1.274939
2	0.581809
4	0.333167
8	0.260673
12	0.257184
20	0.270761
50	0.304380

Tốc độ tăng tốc (*speedup*) được tính theo biểu thức:

$$S = \frac{T_{\text{seq}}}{T_{\text{par}}}$$

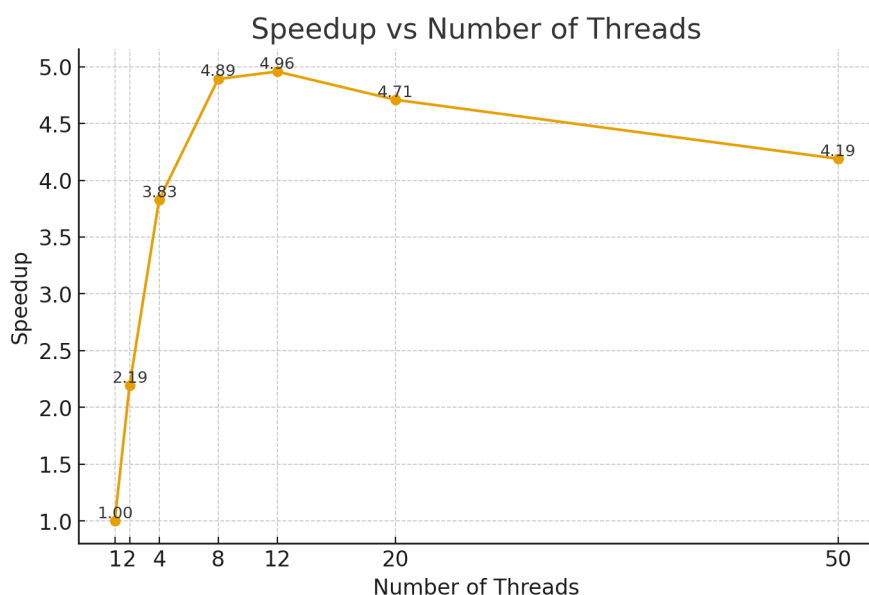


Figure 3: Biểu đồ speed up với số lượng thread khác nhau

Biểu đồ speedup thu được cho thấy xu hướng tăng tốc rõ rệt khi tăng số luồng từ 1 đến 12, với tốc độ tối đa đạt xấp xỉ $5\times$.

Khi số luồng tăng từ 2 lên 8, tốc độ chạy cải thiện rất tốt (speedup đạt từ $2.19\times$ đến $4.89\times$). Điều này phản ánh hiệu quả của mô hình *work pool* trong việc phân phối công việc động: mỗi worker luôn rảnh là lập tức lấy tác vụ tiếp theo, giúp hạn chế thời gian chờ và cân bằng tải tốt.

Tuy nhiên, sau khi vượt qua ngưỡng 12 luồng, hiệu năng bắt đầu bão hoà và giảm dần. Nguyên nhân là do các yếu tố:

- **Tranh chấp tài nguyên** ở hàng đợi tác vụ (mutex contention).
- **Chi phí đồng bộ** tăng lên khi nhiều luồng liên tục truy cập hàng đợi.
- **Giới hạn băng thông bộ nhớ**: khi số luồng lớn hơn số lõi vật lý, dữ liệu truy cập liên tục gây giảm hiệu suất cache.

- **Context switching** gia tăng khi số luồng vượt quá số lõi.

Do đó, tốc độ tăng tốc lớn nhất đạt được trong khoảng 8–12 luồng, tương ứng với cấu trúc phần cứng của bộ xử lý. Khi tăng số luồng lên 20 hoặc 50, hiệu năng giảm nhẹ xuống còn khoảng $4.2\times$, phù hợp với dự đoán từ định luật Amdahl.

Tóm lại, mô hình *work pool* cho thấy khả năng mở rộng tốt trong phạm vi từ 1 đến 12 luồng và đạt hiệu năng tối ưu ở khoảng này. Đây là minh chứng rõ ràng cho việc phân chia công việc động mang lại hiệu quả cao đối với các bài toán mà khối lượng tính toán thay đổi theo không gian như mô phỏng lan truyền sóng xung kích.

References

- [1] Viblo Article. *Tạo một Worker Pool (Thread Pool) đơn giản với Golang*. 2019. URL: <https://viblo.asia/p/tao-mot-worker-pool-thread-pool-don-gian-voi-golang-V3m5WxJwK07>.
- [2] Pascal Baumann et al. “Context-Bounded Verification of Thread Pools”. In: *arXiv preprint* (2021). eprint: [2111.09022](https://arxiv.org/abs/2111.09022).
- [3] Hemant Kumar. *Understanding Thread Pools, Worker Threads and Types of Thread Pools*. 2024. URL: <https://www.tothenew.com/blog/understanding-thread-pools-worker-threads-and-types-of-thread-pools/>.
- [4] KungFuTech.edu.vn. *Thread Pools trong Python*. 2023. URL: <https://kungfutech.edu.vn/bai-viet/python/thread-pools-trong-python>.
- [5] Barak Shoshany. “A C++17 Thread Pool for High-Performance Scientific Computing”. In: *arXiv preprint* (2021). eprint: [2105.00613](https://arxiv.org/abs/2105.00613).
- [6] Unknown. “Thread Pools – Scalable Multithreaded Programming with Thread Pools”. In: *MSDN Magazine* (2010).